

MODULE 34 · EXAM STRATEGY - ALL DOMAINS

Practice exam 1

25 questions across all seven domains. Four options each, as on the real paper.

QUESTIONS 25 FORMAT 4 options, A-D MODULE 34 of 36

How to take this paper

- ◆ **Each question is read aloud, then the slide holds** while you decide.
- ◆ **Pause if you need longer.** The hold is a prompt, not a limit.
- ◆ **Answer every question,** even when unsure. There is no penalty for a wrong answer.
- ◆ **The worked answer follows immediately,** with a reason for each wrong option.

ON THE EXAM

The real exam is 45 questions in 90 minutes — two minutes each. Try to keep near that pace.

A security review asks where notebook source and job configuration are physically held, and where data is actually processed.

The workspace uses serverless compute for all its jobs.

Which statement is correct?

- ☐ A Everything including data processing runs in the control plane
- ☐ B The control plane holds notebook source and job configuration; serverless compute runs in a Databricks-managed layer in the same region
- ☐ C Serverless compute may run in any region, which is what makes it cheaper
- ☐ D Notebook source and data processing both sit in the customer's cloud account

◆ PAUSE AND DECIDE

Which statement is correct?

- A

 Everything including data processing runs in the control plane
The control plane does not process data. That is the compute plane, classic or serverless.
- B

 The control plane holds notebook source and job configuration; serverless compute runs in a Databricks-managed layer in the same region
Correct. The control plane holds notebooks, query definitions and job configuration. Serverless compute is Databricks-managed and regionally co-located.
- C

 Serverless compute may run in any region, which is what makes it cheaper
Serverless runs in the same cloud region as the workspace's classic compute plane.
- D

 Notebook source and data processing both sit in the customer's cloud account
Serverless compute runs in a Databricks-managed layer, not the customer's account.

THE SIGNAL

Two planes, two questions. Where the definitions live, and where the processing happens.

A team runs fourteen scheduled production jobs, all of them on an existing all-purpose cluster that they also use interactively during the working day.

Finance queries why compute spend is high relative to the amount of processing being done.

What is the principal cause?

- ☐ A Scheduled jobs cannot autoscale on all-purpose compute
- ☐ B Fourteen jobs on one cluster causes contention that raises DBU consumption
- ☐ C All-purpose clusters cannot use spot instances for workers
- ☐ D All-purpose compute bills at a higher DBU rate and does not terminate when a job ends

◆ PAUSE AND DECIDE

What is the principal cause?

- A

Scheduled jobs cannot autoscale on all-purpose compute

Autoscaling is available. The cost difference is the billing rate and the termination behaviour.
- B

Fourteen jobs on one cluster causes contention that raises DBU consumption

Contention affects runtime, not the DBU rate. Each job would still be billed at the higher rate.
- C

All-purpose clusters cannot use spot instances for workers

Spot is available for workers on all-purpose compute, so this is not the cause.
- D

All-purpose compute bills at a higher DBU rate and does not terminate when a job ends

Correct. The rate difference is roughly two to three times, and the cluster terminates on inactivity rather than on job completion.

THE SIGNAL

Databricks documents four reasons against running jobs on all-purpose compute. Rate and termination are the two that show up on a bill.

A supplier re-sends a corrected file under exactly the same name as one delivered yesterday, overwriting it in the landing location. The ingestion job uses COPY INTO.

The following morning the corrections have not appeared in the bronze table, and the job reported success.

Why?

- ☐ A The file arrived after the job started, so it will be picked up on the next run
- ☐ B COPY INTO tracks files by identity, not by content or timestamp, so the file was skipped as already loaded
- ☐ C COPY INTO only loads files matching the PATTERN clause, and the name changed
- ☐ D COPY INTO requires `FORMAT_OPTIONS('mergeSchema'='true')` to reload a changed file

◆ PAUSE AND DECIDE

Why?

- A** The file arrived after the job started, so it will be picked up on the next run
It would then be loaded the following night. The scenario states the corrections never appeared.
- B** COPY INTO tracks files by identity, not by content or timestamp, so the file was skipped as already loaded
Correct. Idempotency is by file identity. A modified file with the same name is still considered loaded.
- C** COPY INTO only loads files matching the PATTERN clause, and the name changed
The name is stated as identical, and no pattern change is mentioned.
- D** COPY INTO requires FORMAT_OPTIONS('mergeSchema='true') to reload a changed file
mergeSchema reconciles differing schemas. It has nothing to do with whether a file is reloaded.

THE SIGNAL

'Exactly the same name' is the deciding phrase. COPY_OPTIONS('force='true') is what reloads it.

An Auto Loader stream ingests JSON from a partner who adds new optional fields without notice, several times a month. The stream feeds an operational dashboard that must not go down.

The team wants any unexpected fields preserved for later inspection rather than lost, and they do not want to be woken up.

Which schema evolution mode fits?

- ☐ A failOnNewColumns
- ☐ B none
- ☐ C rescue
- ☐ D addNewColumns

◆ PAUSE AND DECIDE

Which schema evolution mode fits?

- A

 failOnNewColumns
Fails and will not restart until the schema is updated or the file removed. The worst option for uptime.
- B

 none
Ignores new columns entirely, so the data is neither surfaced nor preserved.
- C

 rescue
Correct. The schema never evolves and the stream never fails; unexpected fields land in _rescued_data for inspection.
- D

 addNewColumns
The default when no schema is provided. It fails the stream on a new column and resumes on restart, so the dashboard goes down each time.

THE SIGNAL

Two requirements: never fail, and never lose the data. Only rescue satisfies both.

A team must ingest from Salesforce into bronze. They have no unusual transformation requirements, and they want the least operational burden.

An engineer proposes writing a Structured Streaming job against the Salesforce REST API, because it gives full control.

What should they do instead?

- ☐ A Write the Structured Streaming job as proposed, since it offers the most control
- ☐ B Use Lakehouse Federation to query Salesforce in place
- ☐ C Use Auto Loader against the Salesforce API
- ☐ D Use the managed Salesforce connector in Lakeflow Connect

◆ PAUSE AND DECIDE

What should they do instead?

- A

 Write the Structured Streaming job as proposed, since it offers the most control
Most control and most maintenance: auth, CDC, edge cases, API changes and retries all become yours.
- C

 Use Auto Loader against the Salesforce API
Auto Loader reads files in cloud object storage. An API is the wrong source type for it.
- B

 Use Lakehouse Federation to query Salesforce in place
Federation queries without copying. The requirement is to ingest into bronze.
- D

 Use the managed Salesforce connector in Lakeflow Connect
Correct. Start at the most managed layer and drop down only when a requirement forces it. Salesforce has a managed connector.

THE SIGNAL

Three layers, most managed to least. The rule is to start at the top and drop down only when forced.

A JDBC read from an Oracle database is configured with partitionColumn set to order_id, lowerBound 1000, upperBound 5000 and numPartitions 8. The source table holds order_ids from 1 to 900,000.

An engineer expects roughly 4,000 rows to be returned and is surprised when all 900,000 arrive.

Why?

- ☐ A lowerBound and upperBound set the partitioning stride only; they do not filter rows
- ☐ B The bounds are ignored unless fetchSize is also set
- ☐ C Oracle does not support partitioned JDBC reads, so the bounds were discarded
- ☐ D numPartitions must equal 1 for bounds to be applied

◆ PAUSE AND DECIDE

Why?

- A

 lowerBound and upperBound set the partitioning stride only; they do not filter rows
Correct. They determine how Spark splits the range across partitions. Every row is still returned.
- B

 The bounds are ignored unless fetchSize is also set
fetchSize controls how many rows are retrieved per round trip. It does not filter.
- C

 Oracle does not support partitioned JDBC reads, so the bounds were discarded
Partitioned reads work against Oracle. The behaviour described is the documented one.
- D

 numPartitions must equal 1 for bounds to be applied
Invented rule. numPartitions controls parallelism, not filtering.

THE SIGNAL

A very common misreading. To filter, use a WHERE clause in the query or dbtable option.

A one-off migration must load 1,400 Parquet files from an archive location into a new bronze table. The schema is fixed and well known, the files will never change, and the team works mostly in SQL.

An engineer proposes configuring an Auto Loader stream with a checkpoint and a schema location.

What is the better choice?

- ☐ A Auto Loader as proposed, because it is always the recommended ingestion method
- ☐ B A Lakeflow Connect managed connector
- ☐ C Lakehouse Federation over the archive location
- ☐ D COPY INTO, which suits thousands of files with a known schema and needs no checkpoint or schema location

◆ PAUSE AND DECIDE

What is the better choice?

- A

 Auto Loader as proposed, because it is always the recommended ingestion method
Auto Loader suits continuous arrival at scale. Here it adds a checkpoint and schema location to maintain for a load that runs once.
- B

 A Lakeflow Connect managed connector
Managed connectors cover specific SaaS and database sources. This is a file load from object storage.
- C

 Lakehouse Federation over the archive location
Federation queries external databases in place. It is not a file ingestion mechanism.
- D

 COPY INTO, which suits thousands of files with a known schema and needs no checkpoint or schema location
Correct. This is a bounded, one-off load. COPY INTO is idempotent by default and adds no ongoing state.

THE SIGNAL

Thousands of files, fixed schema, runs once. Auto Loader's machinery is for data that keeps arriving.

A silver build has run nightly for three years. It reads a text column called quantity and casts it to INT. Last week the platform team upgraded the job cluster from Databricks Runtime 15.4 LTS to 17.3 LTS.

The job now fails with CAST_INVALID_INPUT. The upstream data has not changed.

What is the correct response?

- ☐ A Recognise that those rows were previously becoming nulls silently, and use try_cast where null is the correct meaning
- ☐ B Pin the job to Databricks Runtime 15.4 LTS permanently
- ☐ C Set spark.sql.ansi.enabled to false permanently, restoring the previous behaviour
- ☐ D Change the target column to DOUBLE, which accepts a wider range of values

◆ PAUSE AND DECIDE

What is the correct response?

- A** Recognise that those rows were previously becoming nulls silently, and use `try_cast` where null is the correct meaning
Correct. ANSI mode is on by default from DBR 17. The errors are new information about data that was always bad.
- B** Pin the job to Databricks Runtime 15.4 LTS permanently
Freezes the platform to preserve a bug, forfeiting later improvements and security fixes.
- C** Set `spark.sql.ansi.enabled` to false permanently, restoring the previous behaviour
Restores the silent corruption that was happening for three years. Defensible as a temporary unblock, wrong as the fix.
- D** Change the target column to `DOUBLE`, which accepts a wider range of values
`CAST_INVALID_INPUT` means the text is not numeric at all. A wider numeric type parses it no better.

THE SIGNAL

'The upstream data has not changed' is the tell. The data was always like this; the default changed.

A silver orders table with 4.1 million rows is joined to a product_variants table on product_id using an inner join, to attach a variant description.

The result has 6.8 million rows, and total order value calculated from it is around 66% higher than from orders alone.

What is the cause?

- ☐ A The join should have been a left join to preserve the correct count
- ☐ B Some products have several variants, so those orders are duplicated by the join
- ☐ C Order value is being summed from both tables
- ☐ D Null product_ids matched multiple variant rows

◆ PAUSE AND DECIDE

What is the cause?

- A

 The join should have been a left join to preserve the correct count
A left join keeps unmatched rows, which would produce even more rows. This is growth, not shrinkage.
- B

 Some products have several variants, so those orders are duplicated by the join
Correct. Fan-out. An inner join discards unmatched rows, so growth can only come from duplication on one side.
- C

 Order value is being summed from both tables
A variants table would not carry order value, and a duplicated column raises an ambiguous reference rather than inflating a sum.
- D

 Null product_ids matched multiple variant rows
Backwards. Null keys never match anything, so they would reduce the row count.

THE SIGNAL

An inner join that grows the row count is the signature of fan-out. Inner joins normally shrink data.

A silver customer table is built by deduplicating bronze on `customer_id` using `dropDuplicates`. Bronze holds several versions of some customers, because corrections arrive as new rows.

Analysts report that a customer's marketing consent flag sometimes flips between runs, with no upstream change.

What is happening?

- ☐ A `distinct` should be used instead, as it is deterministic
- ☐ B `dropDuplicates` keeps the first row encountered, so bronze must be sorted first
- ☐ C `dropDuplicates` on a subset keeps an arbitrary row per key, so a different version wins on different runs
- ☐ D The consent flag is being rescued into `_rescued_data` on some runs

◆ PAUSE AND DECIDE

What is happening?

- A** distinct should be used instead, as it is deterministic
distinct compares whole rows, so differing versions of a customer are all kept. It does not deduplicate on the key.
- C** dropDuplicates on a subset keeps an arbitrary row per key, so a different version wins on different runs
Correct. There is no guarantee of first, newest or most complete. Use row_number over a window ordered by the update timestamp.
- B** dropDuplicates keeps the first row encountered, so bronze must be sorted first
There is no first-row guarantee, and sorting beforehand does not create one.
- D** The consent flag is being rescued into _rescued_data on some runs
Rescued data applies to schema evolution in Auto Loader, not to deduplication.

THE SIGNAL

Same input, different output, is non-determinism. Something is choosing arbitrarily.

A materialised view aggregates a large managed silver table and refreshes hourly. Only about 0.2% of source rows change each hour, but each refresh consumes roughly the compute of a full rebuild.

Task durations across the refresh are even, and no errors are reported.

What is the most likely cause?

- ☐ A Materialised views always fully recompute
- ☐ B Hourly is too frequent; the refresh should be daily
- ☐ C The refresh is being run with the FULL keyword by the scheduler
- ☐ D Row tracking is not enabled on the source table, so incremental refresh is not possible

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

 Materialised views always fully recompute
Incremental refresh is a documented materialised view feature. It simply has requirements.
- B

 Hourly is too frequent; the refresh should be daily
Changes how often you pay, not what each refresh costs, and it makes the data staler.
- C

 The refresh is being run with the FULL keyword by the scheduler
Possible, and a SCHEDULE clause does not add FULL by default. Someone would have had to write it deliberately.
- D

 Row tracking is not enabled on the source table, so incremental refresh is not possible
Correct. Incremental refresh from Delta tables requires row tracking. Without it Spark falls back to a full recompute, silently.

THE SIGNAL

The fallback is silent. An MV that cannot refresh incrementally does not fail — it recomputes and sends the bill.

A pipeline defines an expectation on a streaming table: `CONSTRAINT positive_qty EXPECT (quantity > 0)`. It was deployed a month ago and the pipeline has run cleanly since.

A downstream analyst reports negative quantities in the silver table. The pipeline's Data quality tab shows several hundred failing records per day.

What is happening?

- ☐ A With no `ON VIOLATION` clause the action is warn, so invalid records are written to the target and only counted
- ☐ B Expectations only apply to records that arrive after deployment
- ☐ C The expectation syntax is invalid and is being ignored
- ☐ D `ON VIOLATION DROP ROW` is the default and is failing to drop

◆ PAUSE AND DECIDE

What is happening?

- A** With no ON VIOLATION clause the action is warn, so invalid records are written to the target and only counted
Correct. Retaining invalid records is the documented default. The expectation is measuring, not enforcing.
- B** Expectations only apply to records that arrive after deployment
Would not explain several hundred currently failing records being written each day.
- C** The expectation syntax is invalid and is being ignored
The Data quality tab is reporting failing counts, which means it is being evaluated correctly.
- D** ON VIOLATION DROP ROW is the default and is failing to drop
Drop is not the default. Warn is, and warn writes the record.

THE SIGNAL

The failing count in the UI is evidence the expectation works. Warn means observe, not enforce.

A job runs on a cluster with 320 total cores. Its longest stage shows exactly 200 tasks, and most executors sit idle throughout it. Task durations within that stage are even.

An engineer sets `spark.default.parallelism` to 640 and re-runs. Nothing changes.

Why, and what should be changed?

- ☐ A AQE overrode the setting and coalesced back to 200
- ☐ B The value must be a power of two, so 640 was rejected
- ☐ C `spark.default.parallelism` applies to RDD operations only; set `spark.sql.shuffle.partitions` instead
- ☐ D The cluster needs more workers before extra parallelism can be used

◆ PAUSE AND DECIDE

Why, and what should be changed?

- A** AQE overrode the setting and coalesced back to 200
AQE coalesces downwards from the configured value. It did not raise anything here, and 200 is the untouched default.
- B** The value must be a power of two, so 640 was rejected
Invented rule. Spark accepts arbitrary values.
- C** spark.default.parallelism applies to RDD operations only; set spark.sql.shuffle.partitions instead
Correct. A DataFrame or SQL shuffle takes its partition count from spark.sql.shuffle.partitions. Exactly 200 tasks is that default showing through.
- D** The cluster needs more workers before extra parallelism can be used
320 cores are already available and idle. Capacity is not the constraint.

THE SIGNAL

Exactly 200 is a default showing through. A round, unchanging number in the Spark UI rarely means coincidence.

A job has four tasks. Task A truncates and reloads a staging table. Tasks B and C read that staging table and write two different silver tables. Task D reads both silver tables.

B declares a dependency on A. C declares none. D depends on B and C. Roughly once a fortnight, one silver table is short of rows.

What is wrong?

- ☐ A C has no dependency on A, so it may read the staging table while A is still loading it
- ☐ B D should depend only on B, since B already depends on A
- ☐ C Task C needs a retry policy to handle the intermittent shortfall
- ☐ D The staging table should be a view rather than a table

◆ PAUSE AND DECIDE

What is wrong?

- A

 C has no dependency on A, so it may read the staging table while A is still loading it
Correct. Undeclared means parallel. C starts alongside A and sometimes wins the race.
- B

 D should depend only on B, since B already depends on A
Would let D run before C finishes, making things worse and more often.
- C

 Task C needs a retry policy to handle the intermittent shortfall
C is not failing. It succeeds with incomplete input, which retries cannot detect.
- D

 The staging table should be a view rather than a table
Does not address the ordering. A view over data still being written has the same problem.

THE SIGNAL

'Roughly once a fortnight' is the tell. A missing dependency fails on timing, not consistently.

A job creates a temporary compute-heavy staging dataset, runs four analyses against it, then deletes it. The delete task depends on all four analyses.

When any analysis fails, the staging dataset is left behind. When somebody cancels the run, it is also left behind.

Which Run if condition should the delete task use?

- | | |
|--|--|
| ☐ A All succeeded | ☐ B None failed |
| ☐ C All done | ☐ D At least one failed |

◆ PAUSE AND DECIDE

Which Run if condition should the delete task use?

A All succeeded

The current behaviour and the cause of the problem. When an analysis fails, the condition is unmet and the delete never runs.

B None failed

Still requires that nothing failed, so it behaves like the current setting in exactly the case that matters.

C All done

Correct. It runs once every dependency has finished, whatever the outcome, and cancellation is covered too.

D At least one failed

Inverts the problem: the dataset would be deleted only when something failed, and left behind on every successful run.

THE SIGNAL

Cleanup should not depend on the outcome of the work it is cleaning up after. All done also covers cancellation.

An ingestion job writes a silver table and normally finishes by 03:40. A downstream aggregation job is scheduled at 04:15. Once or twice a month the ingestion overruns and the aggregation reads yesterday's data, producing wrong figures with no error. Both tables are Unity Catalog managed tables in the same catalog.

What is the best fix?

- ☐ A Move the aggregation schedule to 05:30 for more margin
- ☐ B Use a table update trigger on the silver table so the aggregation starts when it is written
- ☐ C Add a retry policy to the aggregation job
- ☐ D Set maximum concurrent runs on the aggregation job to 1

◆ PAUSE AND DECIDE

What is the best fix?

- A

 Move the aggregation schedule to 05:30 for more margin
A bigger guess is still a guess. It delays the data every day to protect against a monthly event, and will eventually overrun again.
- B

 Use a table update trigger on the silver table so the aggregation starts when it is written
Correct. The dependency becomes explicit rather than being encoded as two cron times 35 minutes apart.
- C

 Add a retry policy to the aggregation job
The aggregation is not failing. It succeeds and produces wrong figures, which retries cannot detect.
- D

 Set maximum concurrent runs on the aggregation job to 1
Governs overlapping runs of the same job. It says nothing about whether upstream data is ready.

THE SIGNAL

A dependency encoded as two cron expressions is a dependency expressed as a hope.

A job takes a parameter called environment, defaulting to dev. The team relies on that default so that only their deployment pipeline, which passes prod explicitly, can write to production.

An analyst who needs to re-run failed jobs has been granted CAN MANAGE RUN.

What risk does this create?

- ☐ A None; the analyst cannot change parameters with CAN MANAGE RUN
- ☐ B The analyst becomes the run-as identity, gaining the job's data access
- ☐ C The analyst could change the job's schedule and cause unexpected runs
- ☐ D The analyst can override the parameter on a manual run; a default parameter value is not a security control

◆ PAUSE AND DECIDE

What risk does this create?

- A

 None; the analyst cannot change parameters with CAN MANAGE RUN
CAN MANAGE RUN or higher can override parameter values for a manual run, which is documented behaviour.
- B

 The analyst becomes the run-as identity, gaining the job's data access
The run-as identity is a separate setting and is not changed by who triggers a run.
- C

 The analyst could change the job's schedule and cause unexpected runs
Changing configuration requires CAN MANAGE. CAN MANAGE RUN allows triggering, stopping and restarting.
- D

 The analyst can override the parameter on a manual run; a default parameter value is not a security control
Correct. The only protection is restricting who holds CAN MANAGE RUN on the job.

THE SIGNAL

The team built a security control out of a convenience feature. Defaults exist so a job runs sensibly, not to stop anyone changing them.

A team keeps all pipeline notebooks in a Git folder. They promote to production by pointing a production-workspace Git folder at main, and syncing after each merge.

They now need production jobs to use a different catalog, larger clusters and a nightly schedule that development does not have.

What should they move to?

- ☐ A Environment-detection logic inside each notebook
- ☐ B A separate Git repository per environment
- ☐ C A separate branch per environment, each with its own hard-coded settings
- ☐ D Declarative Automation Bundles, which package jobs and configuration with the code and support per-target variables

◆ PAUSE AND DECIDE

What should they move to?

- A

 Environment-detection logic inside each notebook
Scatters configuration across notebooks, and cannot set cluster size or a job schedule, which live outside the notebook.
- B

 A separate Git repository per environment
The same divergence problem with more ceremony. The code is identical; only the configuration differs.
- C

 A separate branch per environment, each with its own hard-coded settings
Guarantees divergence. Branches never merged drift apart, and every fix must be cherry-picked into each.
- D

 Declarative Automation Bundles, which package jobs and configuration with the code and support per-target variables
Correct. Bundles version the project rather than only the code, with targets and overrides for each environment.

THE SIGNAL

Git folders are a development tool, not a deployment mechanism. They version code and nothing around it.

A release engineer runs `databricks bundle deploy -t prod` from a hotfix branch. The deployment fails, reporting that the current Git branch does not match the branch specified in the target.

A colleague suggests adding `--force` to the pipeline's production deploy step so this can never block a release again.

What should the team do?

- ☐ A Remove the git property from the production target
- ☐ B Merge the hotfix to main and deploy from there, keeping the branch validation
- ☐ C Add `--force` permanently, since urgent fixes should never be blocked
- ☐ D Point the production target's branch at the hotfix branch

◆ PAUSE AND DECIDE

What should the team do?

- A

 Remove the git property from the production target
Removes the check silently rather than overriding it, which is worse because nobody notices afterwards.
- B

 Merge the hotfix to main and deploy from there, keeping the branch validation
Correct. The check exists to stop production being deployed from a branch that has not been reviewed or merged.
- C

 Add --force permanently, since urgent fixes should never be blocked
Turns a safety check into decoration. It still runs, still reports, and can no longer stop anything.
- D

 Point the production target's branch at the hotfix branch
Production would then track a temporary branch, whose deletion leaves production's declared source non-existent.

THE SIGNAL

Urgency is the argument every safety check eventually faces. The question is whether the check was wrong, and here it was right.

A GitHub Actions workflow deploys a bundle to production. It authenticates using a personal access token generated from the lead engineer's account and stored as a repository secret. The team asks whether this is the right long-term arrangement.

What should they change it to?

- ☐ A Store the token in a Databricks secret scope instead of a repository secret
- ☐ B OAuth U2M, so the workflow logs in interactively at deploy time
- ☐ C OAuth M2M with a dedicated service principal, supplied to the workflow as environment variables
- ☐ D Nothing; a PAT in a repository secret is the current recommendation

◆ PAUSE AND DECIDE

What should they change it to?

- A** Store the token in a Databricks secret scope instead of a repository secret
The runner needs credentials before it can reach Databricks, so the scope cannot be read yet. It also keeps the person-owned token.
- B** OAuth U2M, so the workflow logs in interactively at deploy time
U2M requires a browser sign-in. There is nobody in a CI runner to complete it.
- C** OAuth M2M with a dedicated service principal, supplied to the workflow as environment variables
Correct. No interactive sign-in, short-lived tokens, and production deployment belongs to a service principal rather than a person.
- D** Nothing; a PAT in a repository secret is the current recommendation
It works, and it ties production deployment to one person's account and uses the method Databricks now labels legacy.

THE SIGNAL

Two problems, not one: the method is now labelled legacy, and the identity is an individual.

A job's longest stage takes 52 minutes across 400 tasks. Summary Metrics show a median task duration of 31 seconds, a 75th percentile of 38 seconds, and a maximum of 47 minutes. No spill statistics appear on the stage page.

What is the bottleneck, and what should be tried first?

- ☐ A Insufficient memory; move to a memory-optimised node type
- ☐ B Data skew; confirm AQE skew join handling is active before writing any salting logic
- ☐ C Under-parallelism; raise `spark.sql.shuffle.partitions`
- ☐ D The cluster is undersized; add executors so the slow task finishes faster

◆ PAUSE AND DECIDE

What is the bottleneck, and what should be tried first?

A Insufficient memory; move to a memory-optimised node type

The scenario states no spill, which is the symptom memory pressure produces.

C Under-parallelism; raise spark.sql.shuffle.partitions

400 tasks with 399 finishing in seconds is not an under-parallelism problem, and more partitions do not split one hot key.

B Data skew; confirm AQE skew join handling is active before writing any salting logic

Correct. 47 minutes against a 38-second 75th percentile is vastly past the 50% threshold. AQE splits skewed partitions at runtime and is on by default.

D The cluster is undersized; add executors so the slow task finishes faster

Executors are idle, not struggling. One task holds the stage open while the rest have nothing to do.

THE SIGNAL

Databricks' own threshold: if Max duration is 50% more than the 75th percentile, suspect skew. Here it is roughly 74 times.

A nightly job fails with an out-of-memory error. The team doubles worker memory twice, at considerable cost, and it still fails. Reading the stack trace properly, they find it names the driver.

The job ends by collecting the result to build a formatted report.

What should they do?

- ☐ A The failure is on the driver; aggregate to report size before collecting, or write the file without collecting
- ☐ B Continue increasing worker memory until it succeeds
- ☐ C Enable AQE to handle the skew causing the failure
- ☐ D Raise `spark.sql.shuffle.partitions` to reduce partition size

◆ PAUSE AND DECIDE

What should they do?

- A

 The failure is on the driver; aggregate to report size before collecting, or write the file without collecting
Correct. collect and toPandas pull the whole result to a single machine. The driver does not process partitions.
- B

 Continue increasing worker memory until it succeeds
Two doublings have already failed, which is the evidence that workers are not the constraint.
- C

 Enable AQE to handle the skew causing the failure
AQE optimises distributed execution. It has no effect on a single-machine collection to the driver.
- D

 Raise spark.sql.shuffle.partitions to reduce partition size
Shuffle partitions govern executor-side partition size. The driver does not process partitions.

THE SIGNAL

Two failed doublings were already the answer. If more executor memory does not help, the executors were never the problem.

A team drops nine tables during a schema clean-up. Two turn out to be needed. One is restored in minutes by running CREATE TABLE against its original storage path. The other cannot be restored that way.

Both were created by the same engineer, in the same schema, on the same day.

What is the difference between them?

- ☐ A The restorable one was external, so DROP removed only its metadata; the other was managed and its files were deleted
- ☐ B DROP behaves differently depending on how recently a table was queried
- ☐ C The restorable one had been backed up and the other had not
- ☐ D The unrestorable one had predictive optimisation enabled, which vacuumed its files

◆ PAUSE AND DECIDE

What is the difference between them?

- A** The restorable one was external, so DROP removed only its metadata; the other was managed and its files were deleted
Correct. Pointing CREATE TABLE at the original path only works if the files are still there, which is external behaviour.
- B** DROP behaves differently depending on how recently a table was queried
Invented behaviour. DROP does not consider query history.
- C** The restorable one had been backed up and the other had not
The described recovery was recreating a table over an existing path, not restoring from a backup.
- D** The unrestorable one had predictive optimisation enabled, which vacuumed its files
VACUUM removes files not referenced by any table version, on an existing table. DROP deleted these.

THE SIGNAL

The recovery method is the evidence. Pointing CREATE TABLE at a path only works if the files survived the DROP.

A new analyst is added to the reporting group, which holds SELECT on twelve tables in the finance catalog. SHOW GRANTS confirms the grants and names her group correctly.

Every query she runs fails with a permissions error. Colleagues in the same group query the same tables without difficulty.

What is the most likely explanation?

- ☐ A She lacks USE CATALOG on finance or USE SCHEMA on the schema, which her colleagues hold via another group
- ☐ B She needs ALL PRIVILEGES on the tables rather than SELECT
- ☐ C Grants take up to 24 hours to propagate to new group members
- ☐ D A DENY on the catalog is overriding her SELECT

◆ PAUSE AND DECIDE

What is the most likely explanation?

- A** She lacks USE CATALOG on finance or USE SCHEMA on the schema, which her colleagues hold via another group
Correct. Reaching a table requires traversal privileges as well as SELECT. Same group, same grant, different outcome means something held elsewhere.
- B** She needs ALL PRIVILEGES on the tables rather than SELECT
SELECT is the correct privilege for reading. ALL PRIVILEGES is far broader and still would not supply traversal on its own.
- C** Grants take up to 24 hours to propagate to new group members
Invented delay, and it would not explain colleagues succeeding right now.
- D** A DENY on the catalog is overriding her SELECT
Unity Catalog does not support DENY. Its model is least privilege: ungranted is implicitly denied.

THE SIGNAL

Same group, same grant, different outcome. The difference must come from a privilege held outside that group.

A governance team tags every table in a catalog with pii:true, then creates an ABAC column mask policy on that catalog using MATCH COLUMNS has_tag_value('pii','true').

The policy is created without error and SHOW POLICIES confirms it. Nothing is masked for anyone.

Why?

- ☐ A A WHEN clause is required for a column mask policy to take effect
- ☐ B The policy must be attached to each table individually
- ☐ C The tags were applied to tables; column tags do not inherit from the parent table and must be applied directly
- ☐ D has_tag_value is not supported inside MATCH COLUMNS

◆ PAUSE AND DECIDE

Why?

- A** A WHEN clause is required for a column mask policy to take effect
WHEN is optional. It narrows which securables match; omitting it considers all of them.
- B** The policy must be attached to each table individually
Catalog-level attachment is normal and intended. Scope is not the problem.
- C** The tags were applied to tables; column tags do not inherit from the parent table and must be applied directly
Correct. Tag inheritance can be overridden at every level except columns. MATCH COLUMNS finds no column tags, so nothing matches.
- D** has_tag_value is not supported inside MATCH COLUMNS
has_tag and has_tag_value are the only two functions supported in policy conditions, and both work there.

THE SIGNAL

The policy is correct and matches nothing. MATCH COLUMNS looks at column tags, and only tables were tagged.

What this paper covered

DOMAIN	QUESTIONS
Domain 1 – Platform	2
Domain 2 – Ingest	5
Domain 3 – Transform	6
Domain 4 – Jobs	4
Domain 5 – CI/CD	3
Domain 6 – Troubleshoot	2
Domain 7 – Govern	3
Total	25

ON THE EXAM

Weighted across all three papers rather than within each one, so a single paper will not match the exam's weights exactly.